Task 1

Black-Box Reconnaissance & IDOR Discovery

Black-Box Reconnaissance & IDOR Discovery

Prepared for Datheon internship mentors and project reviewers

Prepared 05 November 2026

Review scope and safety notice

This document describes an intentionally vulnerable, loopback-only mock application containing fictional training records. It is not a finding against Datheon, OWASP crAPI, or any production service.

Contents

1 Reviewer at a glance 1
2 Scope and review boundary 1
3 Workflow demonstrated 2
4 Reproduction steps for reviewers 2
5 Observed output from the included mock 2
6 Security finding analysis 3
6.1 Root cause 3
6.2 Potential impact if deployed remotely 3
6.3 CVSS 3.1 assessment — illustrative model only 4
7 Expected remediation 4
8 Visual evidence for reviewers 4
9 Mentor review conclusion 5

1 Reviewer at a glance

This packet gives a mentor or project reviewer a concise, reproducible view of the Datheon internship Task 1 submission. It documents a controlled black-box reconnaissance exercise for the internship brief. The local mock dashboard accepts a client-supplied `userId` query parameter and returns the matching record from an unauthenticated API endpoint. By changing only the identifier in the URL or cURL request, a tester can retrieve a different synthetic record without sending cookies, an `Authorization` header, or any login credentials.

The result demonstrates the **pattern** of an unauthenticated Broken Object Level Authorization (BOLA/IDOR) weakness. The evidence is limited to a purpose-built local mock at `127.0.0.1:5050`; all records are fictional and the server is not externally reachable.

Finding status: Reproduced in the local mock only

Two unauthenticated requests for different synthetic identifiers returned HTTP 200 with different profile records. The modeled remote-impact severity is illustrative only: CVSS 3.1 **5.3 Medium** with vector `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`.

What reviewers can verify A reviewer can start the included loopback mock, run the three cURL commands in this packet, and confirm that changing only the object identifier changes the returned synthetic record while no credentials are supplied. The packet also maps the observation to root cause, impact, CVSS, and an appropriate server-side remediation.

2 Scope and review boundary

Item	Value
Target	Fieldnote local mock dashboard, created for this training exercise
Base URL	`http://127.0.0.1:5050`
Endpoint	`GET /api/dashboard?userId=<demo-id>`
Data	Five static fictional records: IDs 104–108
Tools	Browser DevTools Network tab, cURL, optional Postman/Burp
Authorization boundary	Testing limited to the included local source and loopback service
Out of scope	Datheon production systems, third-party apps, real people, and external hosts

Safety controls The mock binds only to `127.0.0.1`, uses Python's standard library, and marks its records as fake training data. Do not change the bind address, forward the port, deploy the service, or replace the fixtures with real personal information.

3 Workflow demonstrated

The workflow below follows the internship brief: observe the dashboard request, isolate the object identifier, remove credentials, replay the request, compare responses, and document the result without claiming access to a real target.

1. Observe

Open the local dashboard and inspect the Network request.

→

2. Isolate

Identify `userId=104` in the request URL.

→

3. Strip

Remove cookies, Authorization, and session headers.

→

4. Compare

Replay with `userId=105`; compare status and body.

The final conclusion is based on server-side behavior, not on whether the UI displays or hides the identifier. A client can always edit a URL or replay an API request directly.

4 Reproduction steps for reviewers

Start the intentionally vulnerable mock from the Task 1 folder:

```
python3 app.py
```

Then use cURL without cookies or authorization headers. The following command set tests two known synthetic records and one unknown identifier:

```
curl -i 'http://127.0.0.1:5050/api/dashboard?userId=104'
curl -i 'http://127.0.0.1:5050/api/dashboard?userId=105'
curl -i 'http://127.0.0.1:5050/api/dashboard?userId=999'
```

For a browser workflow, open `http://127.0.0.1:5050`, open Developer Tools → Network, reload the page, and inspect `/api/dashboard?userId=104`. Change only the query parameter to `105` and reload. The browser implementation explicitly uses `credentials: 'omit'`, making the absence of authentication visible in the request design.

5 Observed output from the included mock

The following is a representative capture generated by replaying the included local mock on 05 November 2026. The first two requests returned different records even though no credentials were supplied. The third request correctly returned a generic unknown-ID response.

```
### Request 1: no credentials, userId=104
HTTP/1.0 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
{"userId":"104","appId":"DT-DEMO-104","name":"Alex Example",
 "email":"alex@example.test","program":"Application Security Practice",
```

```
 "notes":"FAKE LAB RECORD — no real person or account.",
 "status":"Training record"}

### Request 2: no credentials, userId=105
HTTP/1.0 200 OK
Content-Type: application/json; charset=utf-8
Cache-Control: no-store
{"userId":"105","appId":"DT-DEMO-105","name":"Jordan Sample",
 "email":"jordan@example.test","program":"API Security Practice",
 "notes":"FAKE LAB RECORD — this record is also intentionally exposed.",
 "status":"Training record"}

### Request 3: unknown identifier userId=999
HTTP/1.0 404 Not Found
Content-Type: application/json; charset=utf-8
{"error":"Unknown demo userId. See the README for the local test values."}
```

Request	Status	Security-relevant observation
`userId=104`, no credentials	200	Synthetic record returned
`userId=105`, no credentials	200	Different synthetic record returned by changing only the identifier
`userId=999`, no credentials	404	Unknown identifier is rejected

Evidence interpretation The security signal is the combination of: (1) no authentication material in the request, (2) a client-controlled object identifier, and (3) successful retrieval of a different object after changing only that identifier. The records are synthetic; therefore the exercise proves a vulnerability pattern in the mock, not a production incident.

6 Security finding analysis

6.1 Root cause

The server performs a direct lookup using the `userId` supplied by the caller. It does not establish an authenticated principal and does not compare that principal with the owner of the requested record. Hiding the parameter in the interface would not remediate the issue because the API remains directly callable.

6.2 Potential impact if deployed remotely

In a real system, repeated identifier changes could expose personal profiles, email addresses, application details, notes, or other customer/student data. The practical impact would depend on the sensitivity and volume of the underlying data, identifier predictability, rate limits, monitoring, and whether the endpoint were reachable by an untrusted caller.

6.3 CVSS 3.1 assessment — illustrative model only

Metric	Value	Rationale for a hypothetical remotely reachable service
Attack Vector	Network	A caller can send the HTTP request remotely.
Attack Complexity	Low	Only the object identifier needs to be changed.
Privileges Required	None	The mock accepts the request without login.
User Interaction	None	The attacker can call the endpoint directly.
Scope	Unchanged	Impact remains within the application data owner.
Confidentiality	Low	The modeled issue is unauthorized read access.
Integrity	None	No write operation was demonstrated.
Availability	None	No availability impact was demonstrated.

Illustrative CVSS 3.1 score: 5.3 / 10.0 — Medium

Vector: `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`

This score must not be applied to the actual loopback-only mock. A real assessment would require verified authorization, deployment reachability, data sensitivity, and production evidence.

7 Expected remediation

1. Require authentication for private dashboard and record endpoints.
2. Derive the requesting identity from a validated server-side session or token; never trust a caller-supplied owner identifier.
3. Enforce object-level authorization in the data access query, for example by requiring both the requested record ID and the authenticated owner ID to match.
4. Return `401 Unauthorized` for missing or invalid authentication. For a valid user requesting another user's object, return a privacy-preserving `404 Not Found` or a carefully justified `403 Forbidden`, without record details or database diagnostics.
5. Add integration tests for missing credentials, invalid credentials, owner access, cross-user denial, unknown IDs, and non-disclosure of protected fields.
6. Minimize public response fields, rate-limit repeated identifier guessing, log security events safely, and avoid storing tokens or unnecessary personal data in browser storage.

8 Visual evidence for reviewers

The following browser capture shows the local dashboard as a reviewer would see it. The address bar contains the loopback URL and the `userId=104` query parameter; the page identifies itself as a local, intentionally vulnerable training mock and labels the displayed record as synthetic.

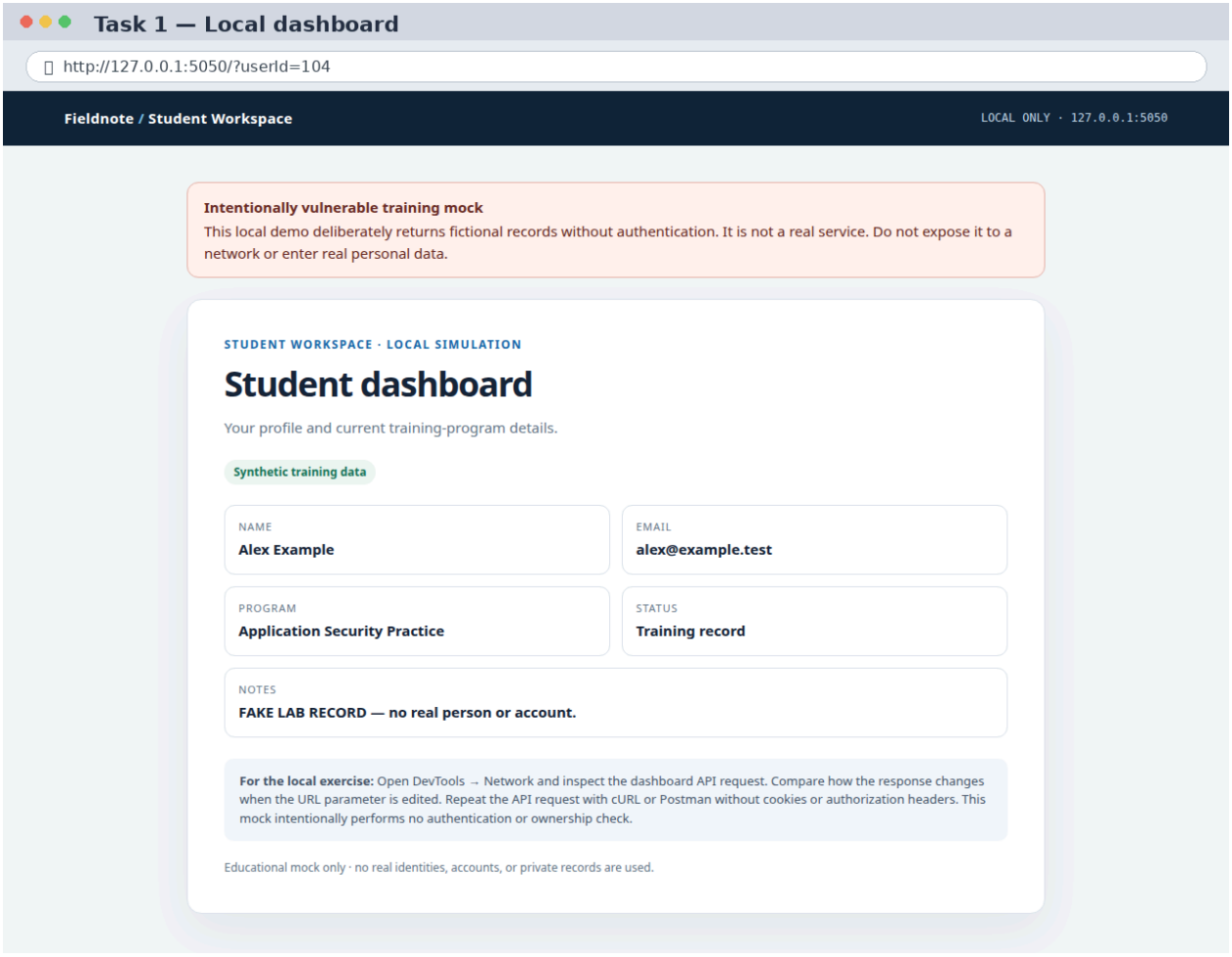


图 1 Browser view of the local mock at `127.0.0.1:5050/?userId=104`.

What this visual evidence demonstrates The dashboard renders a record selected by a URL-controlled object identifier. The accompanying cURL capture in this packet shows that the same API can be called without credentials and that changing the identifier to `105` returns a different synthetic record. No real identity, account, or production service is involved.

9 Mentor review conclusion

The local exercise successfully demonstrates how a black-box tester can identify an IDOR/BOLA pattern using ordinary browser tools and cURL: observe the API request, remove credentials, change the object identifier, and compare the response. The evidence is reproducible and the remediation guidance maps directly to the internship brief.

The responsible conclusion is deliberately narrow: **the included mock is intentionally vulnerable**. It does not establish that Datheon, OWASP crAPI, or any other real application is vulnerable. Keep the demo on loopback, use fictional data only, and obtain explicit authorization before testing any system that is not part of this local lab.

Documentation note: This packet was assembled with AI assistance from the supplied Task 1 source files. The HTTP examples were reproduced from the included local mock and are labeled as synthetic training evidence.